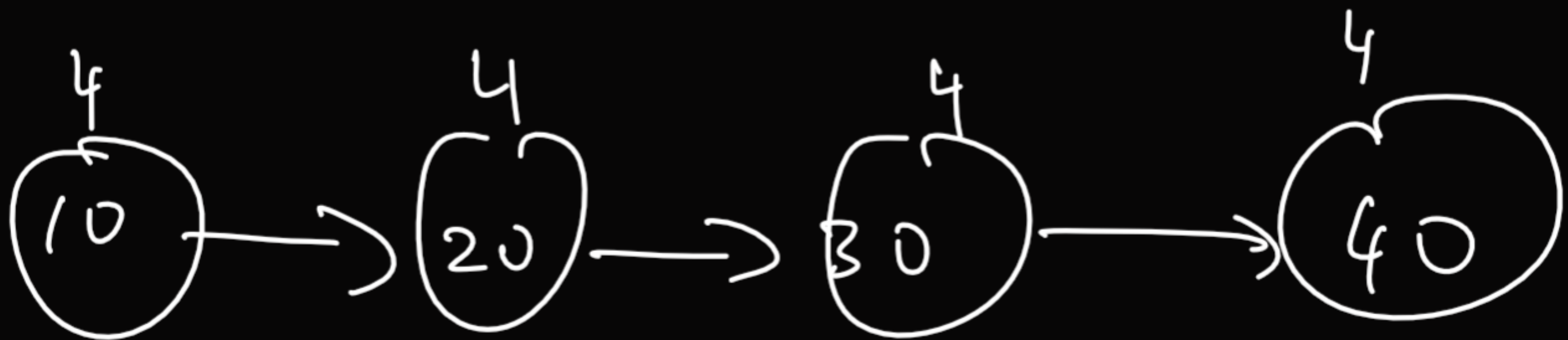
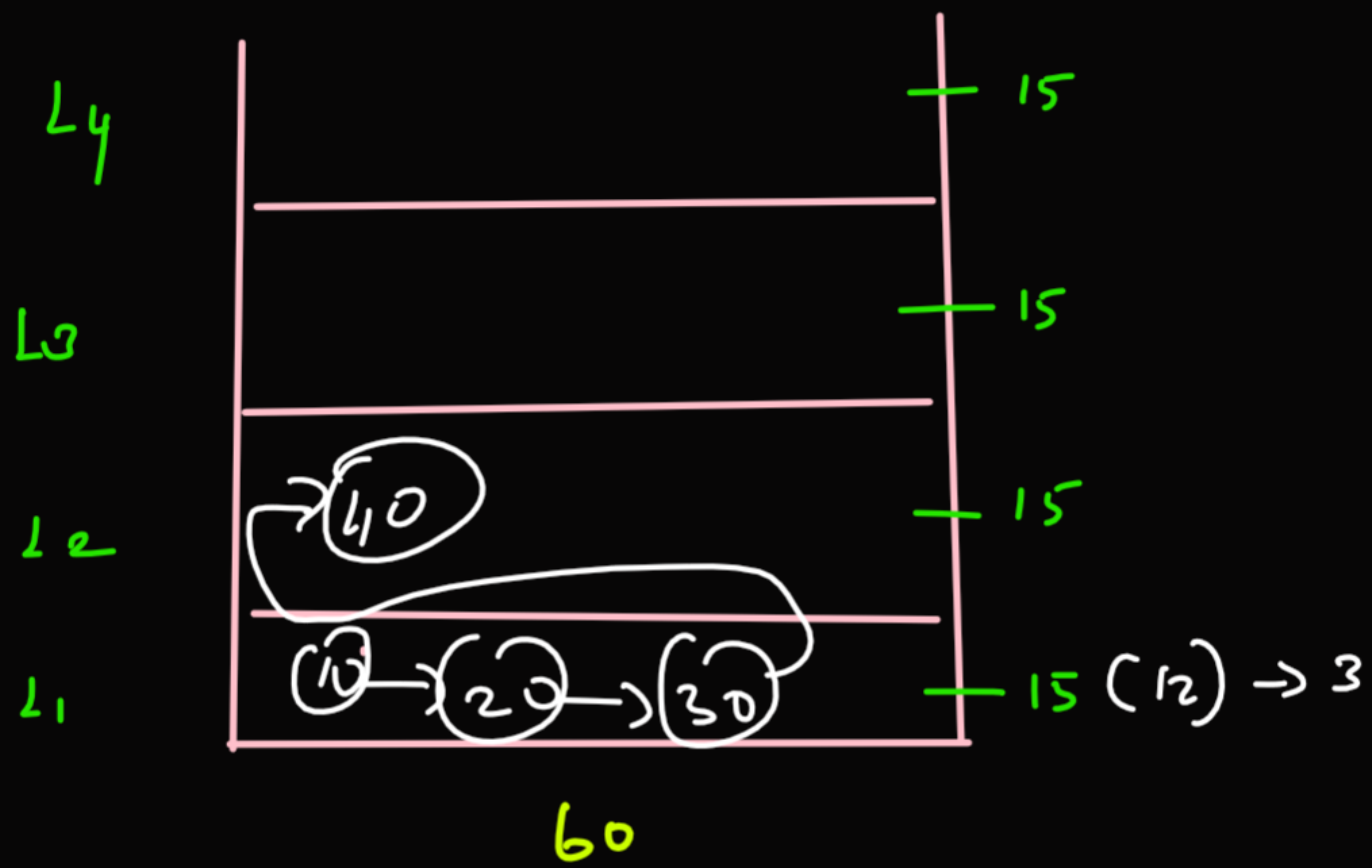


$A = \boxed{20}$



String → group of character
Immutable (can't alter & update)

String str = "naven";

Scanner scn = new Scanner(System.in);

str = scn.next();
scn.nextLine();

welcome to str

0	1	2	3	4	5
n	a	v	e	e	n

 = A

str = "n a v e e n"

str.charAt(3) = e

A[3] = e

str.substring(st, end)
 1 5

String Palindrome

$S_1 =$ " r a c e c a r "

$S_2 = ""$

$S_2 =$ r a c e c a r "

i j
 m a d a m
 0 1 2 3 4
 i j
 0 4
 1 3
 2 2

[a b c d e f g h i j]

no of Iteration = n

Iteration = n

$\frac{n}{2}$

$O(n) \rightarrow$

$O(n) \rightarrow$

$S.C = O(n)$

$S.C = O(1)$

$S_1 = "$

$S_2 =$

LeetCode 2108

```
class Solution {
    public String firstPalindrome(String[] words) {
        for(String word: words){
            if(isPalindrome(word)){
                return word;
            }
        }
        return "";
    }
    boolean isPalindrome(String A){
        int i=0;
        int j=A.length()-1;
        while(i<j){
            if(A.charAt(i)!=A.charAt(j))
                return false;

            i++;
            j--;
        }
        return true;
    }
}
```

Search in a row-col sorted matrix

	0	1	2
0	3	30	38
1	20	52	54
2	35	60	69

$n \times m$

while ($r \geq 0$ && $r < n$ &&
 $c \geq 0$ && $c < m$)

if ($pos < x$)

$r++$;

if ($pos > x$)

$c--$;

if ($pos == x$)

62

r	c
0	2
1	2
2	2
2	1


```
// your code here
int n=A.length;
int m=A[0].length;
int r=0;
int c=m-1;
while(r>=0 && r<n && c>=0 && c<m){
    if(A[r][c]<x){
        r++;
    }else if(A[r][c]>x){
        c--;
    }else{
        return true;
    }
}
return false;
```

Char & Strings Brush up

char ch = 'a', ' ' (circled)

isUpperCase

String str = " "

```
// your code here
int uc=0;
int lc=0;
int d=0;
int sc=0;
for(int i=0;i<s.length();i++){
    char ch=s.charAt(i);
    if(Character.isUpperCase(ch)){
        uc++;
    }else if(Character.isLowerCase(ch)){
        lc++;
    }else if(Character.isDigit(ch)){
        d++;
    }else{
        sc++;
    }
}
int ans[]=new int[4];
ans[0]=uc;
ans[1]=lc;
ans[2]=d;
ans[3]=sc;
return ans;
}
```


String & methods

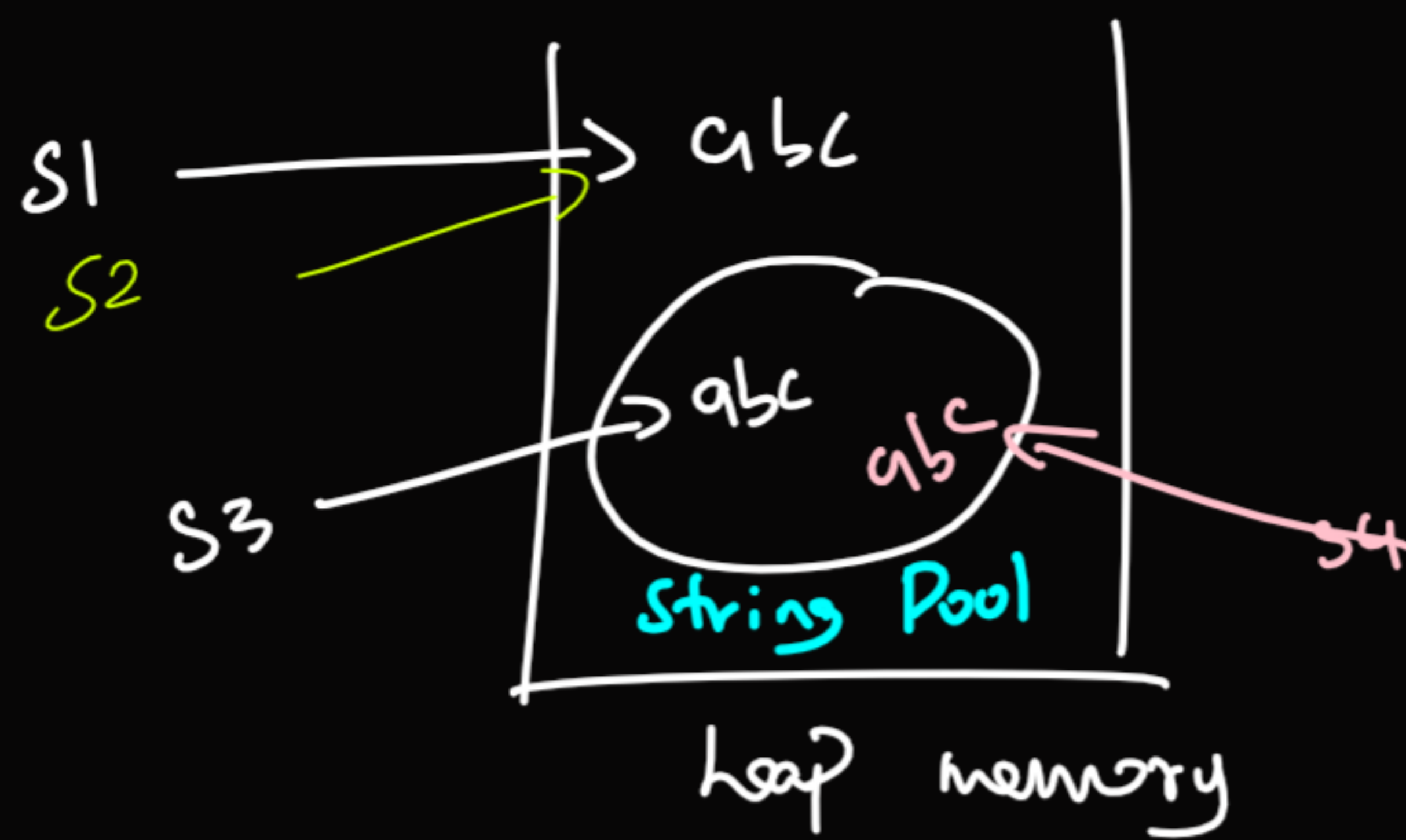
1) Group of characters, immutable.

- 1) charAt $\rightarrow$ char
- 2) length $\rightarrow$ Integer
- 3) substring(st, end) $\rightarrow$ String
- 4) substring(st) $\rightarrow$ String
- 5) replace(s, T) $\rightarrow$ new String with updation
- 6) replaceAll("s", "T") $\rightarrow$ UWS
- 7) toCharArray() $\rightarrow$ char Array
- 8) split(" ") $\rightarrow$ condition
- 9) s1.equals(s2) $\rightarrow$ True } bool
false

Split
"I" "Love" "Programming"

String arr[] = { "I", "Love", "Programming" }

```
String str="I Love Programming";
System.out.println(str.length());
System.out.println(str.substring(3));
System.out.println(str.replaceAll("[ao]", ""));
char[] Arr=str.toCharArray();
String[] splitArr=str.split(" ");
System.out.println(Arr[0]);
```

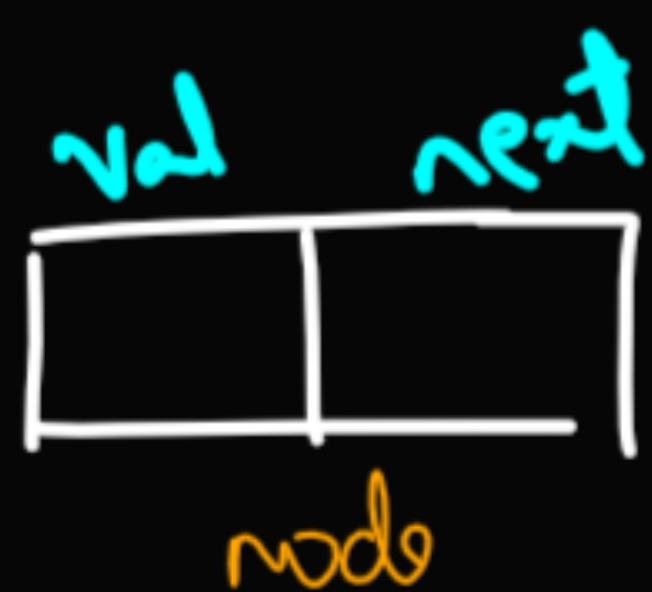



`==` (to compare Address) `s1.equals(s2)` - To compare values

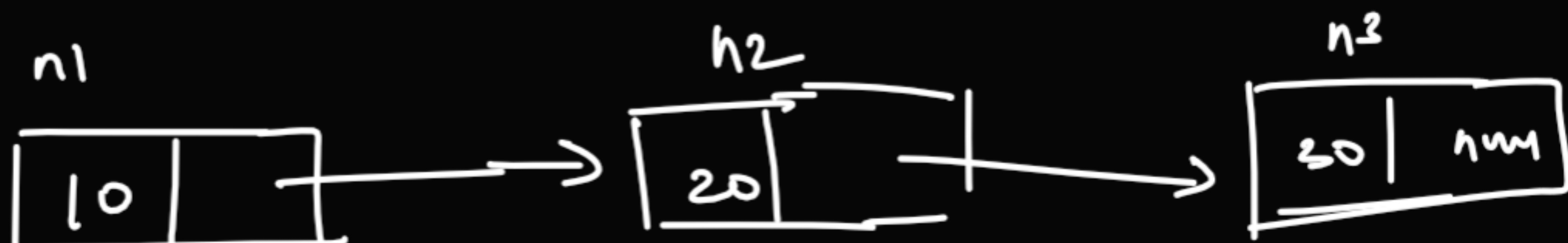
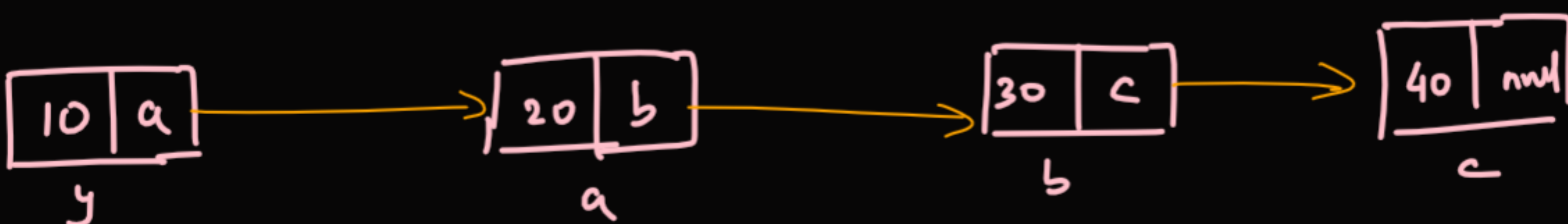
class Intro

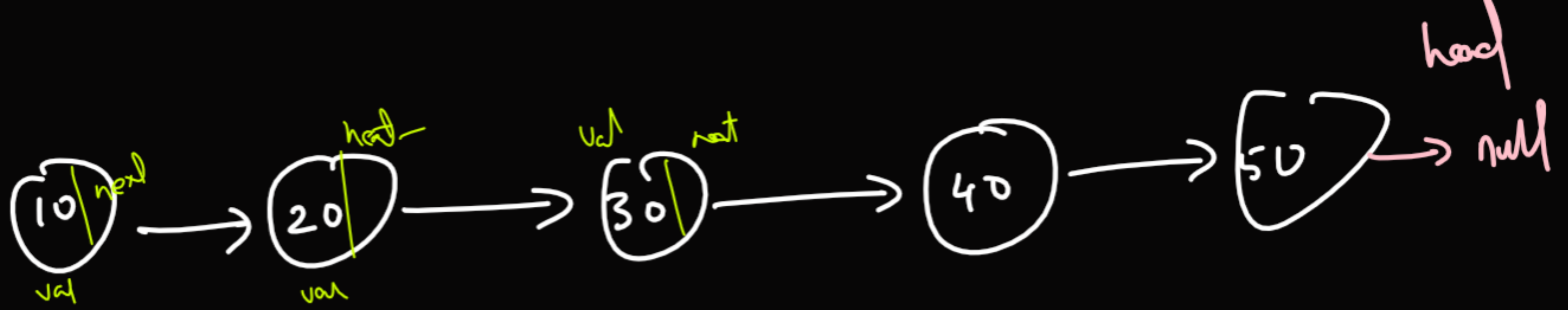
```
class student{
    int age;
    String name;
    student(int age, String name){
        this.age=age;
        this.name=name;
    }
}
```

Linked List



h





while (head != null) {

Print(head.val) — 10, 20

head = head.next

}

Display a Linked list

```
class Solution {  
    // Function to display the elements of a li  
    void printList(Node head) {  
        // add code here.  
        while(head!=null){  
            System.out.print(head.data+" ");  
            head=head.next;  
        }  
    }  
}
```